



# In eigener Sache: Wir schließen im Praktikum bisherige Lücken des MATSE-Rahmenlehrplans

## Übersicht über die Lernfelder für den Ausbildungsberuf

### Mathematisch- technischer Softwareentwickler/-in...

#### Lernfelder

Nr.

#### Zeitrichtwerte in Unterrichtsstunden

1.

2.

3.

Jahr

Jahr

Jahr

11 Parallele Prozesse gestalten und in Netzwerken programmieren

80

Prozesskommunikation

OSI-Referenzmodell

Kommunikation über gemeinsame Variablen und über Verwendung von Nachrichten

Probleme der Kommunikation: Race Condition, Wettlaufsituation

Entwicklung von nebenläufigen Prozessen

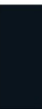
Prozessbeschreibung, Synchronisationsarten und Probleme der Synchronisation: Deadlock, Semaphore, kritische Abschnitte



## Daher: Entwicklung eigener verteilter Anwendungen

---

- Im Praktikum geht es ganz wesentlich um die Entwicklung eigener Anwendungen
  - Vermittlung elementarer Fähigkeiten in Webengineering (Asynchrones Programmieren) und hier (Fokus auf synchrones Programmieren mittels Threads!)
  - Hierzu werden eigene Anwendungen entwickelt, die die Aufgaben der Schichten 5-7 und die Anwendung selber realisieren
  - Basis hierzu ist wird bei uns im **Praktikum** die **Schnittstelle zum Betriebssystem** sein, die **Socket-Schnittstelle**
  - **Hinweis:** Es gibt höher wertige Schnittstellen auf Basis von Middleware-Lösungen
- Schauen wir uns also zunächst in der Vorlesung die Socket-Schnittstelle an
- Die wichtigen Details zu den unteren 4 Schichten folgen dann in späteren Vorlesungen



# Entwicklung eigener verteilter Anwendungen

---

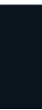
- TCP/UDP (Transportschicht) und IP (Vermittlungsschicht) sind im Betriebssystem implementiert
- Wird eine eigene Anwendung entwickelt, so bauen wir im Praktikum mittels der socket-Schnittstelle direkt auf **TCP oder UDP** auf
- **Port-Nummern identifizieren** hierbei **die Anwendung**, die **IP-Adressen den Zielrechner** in einem Netz aus Netzen (dem Internet)
- Port-Nummern sind 16-Bit-Größen, haben also einen Wertebereich zwischen 0-65535
  - Bei Verwendung der Port-Nummer 0 wird das Betriebssystem einen freien Port wählen
  - Port-Nummern zwischen 1-1023 benötigen Administrator-Privilegien, diese nutzen wir für unsere eigenen Anwendungen nicht (Well-Known-Ports)
- IP-Adressen lernen wir noch, im Praktikum werden wir zunächst eine besondere IP-Adresse nutzen:
  - 127.0.0.1 <-> localhost



# Entwicklung eigener verteilter Anwendungen

---

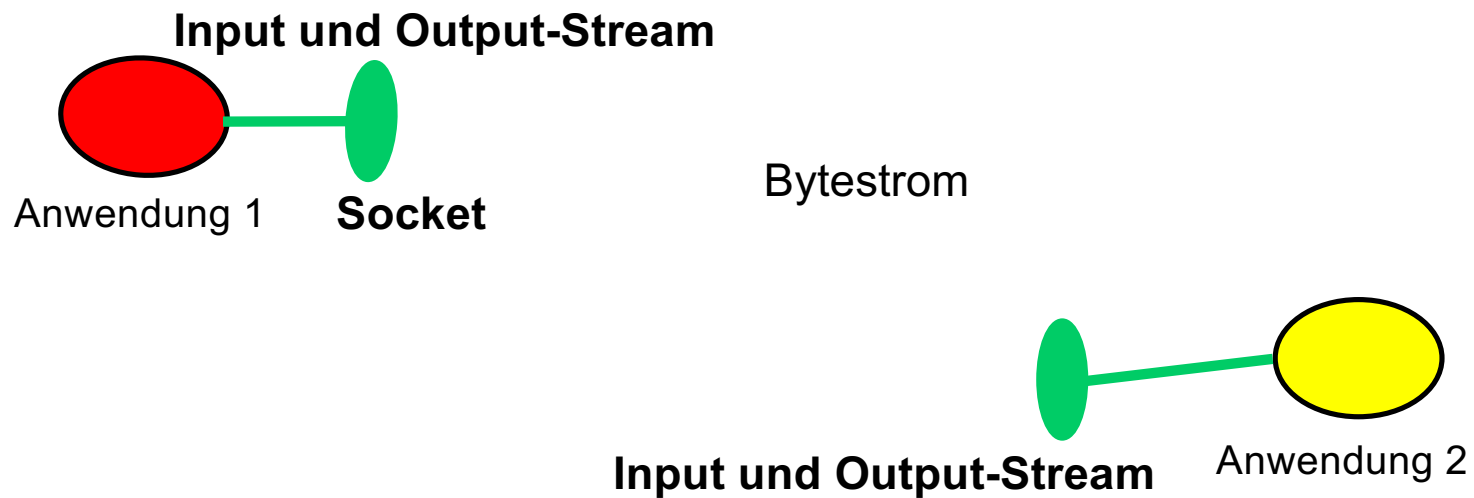
- Sockets:
  - Abstraktion für den Zugang zum Transportprotokoll, die vom Betriebssystem bereitgestellt wird
  - Teilt man einer Instanz dieses Typs Adressinformationen des Kommunikationspartners mit, wird bei TCP ein logischer *Kommunikationskanal* zu diesem Partner aufgebaut. Bei UDP wird anschauliche so etwas wie eine Postkarte bereit gestellt
  - Bei **TCP** kann man einfach durch Schreiben bzw. Auslesen des Sockets Daten übertragen; In Java werden hierzu **Streams** verwendet, die mit dem Socket verknüpft sind
  - Bei **UDP** wird eine Art **Briefkasten** benutzt, der die Nachrichten (mit ausgewiesenem Absender und Empfänger) aufnimmt und (hoffentlich auch) überträgt
- Die/der Programmierer:in ist verantwortlich dafür, was übertragen wird!





## Entwicklung eigener verteilter Anwendungen

- TCP-Sockets: Zwei logische Kommunikationskanäle zwischen den Endanwendungen





## Entwicklung eigener verteilter Anwendungen

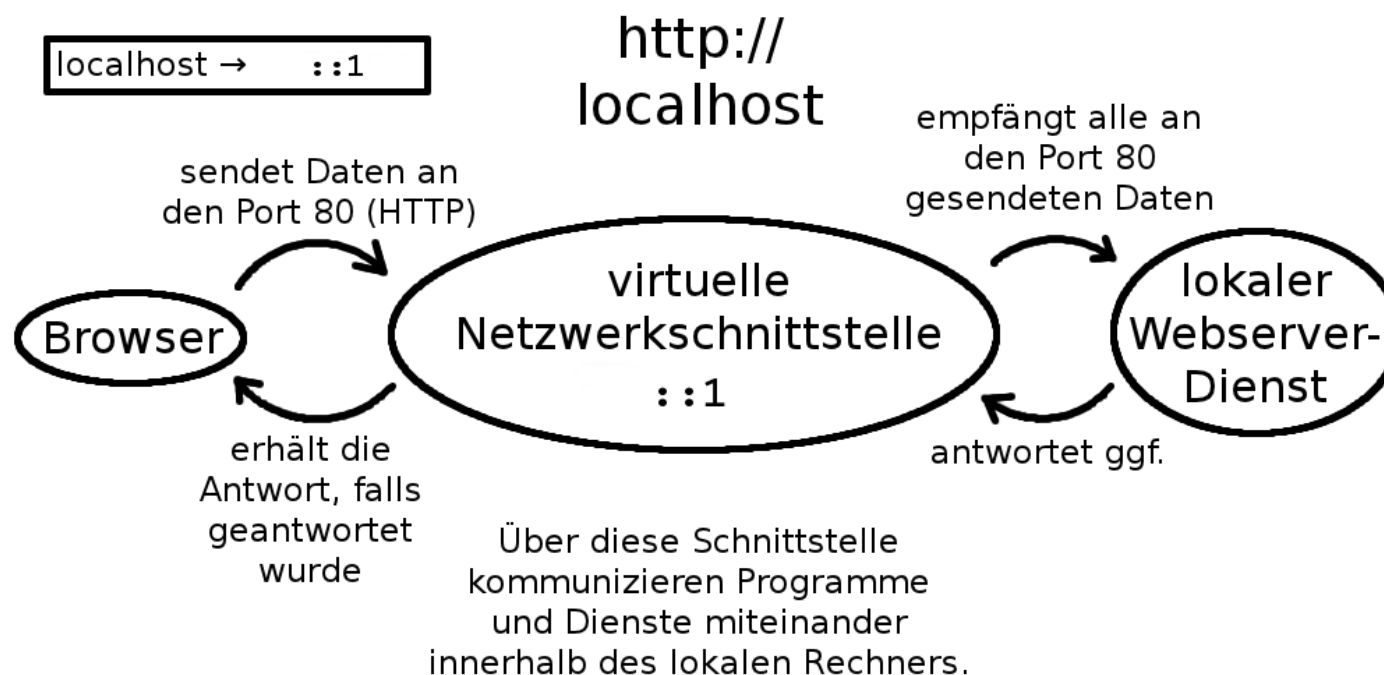
---

- Die IP-Adresse 127.0.0.1 (localhost, loopback)
  - Wir nutzen im Praktikum zunächst eine spezielle IP-Adresse, konkret eine IPv4-Adresse
  - Schon jetzt sei gesagt, dass viele Anwendungen den Nutzern erlauben, als Zieladresse auch Namen zu verwenden (z.B. [www.fh-aachen.de](http://www.fh-aachen.de)). Faktisch findet hierbei eine **automatische Umsetzung** dieser **Namen** auf **IP-Adressen** statt (DNS, hosts-Datei)
  - **localhost** folgt einer Loopback-Idee. **Jeder Rechner hat eine solche Adresse**. Faktisch haben wir hier keine Netzwerkkarte, sondern eine virtuelle Netzwerkschnittstelle mit IP-Adresse. Die virtuelle Netzwerkschnittstelle wird genutzt, um mit sich selbst zu kommunizieren



## Entwicklung eigener verteilter Anwendungen

- Die IP-Adresse 127.0.0.1 (localhost) respektive ::1 (IPv6)



Von [https://de.wikipedia.org/wiki/Benutzer:Alexander\\_Klimov](https://de.wikipedia.org/wiki/Benutzer:Alexander_Klimov) - [https://commons.wikimedia.org/wiki/File:HTTP\\_localhost.png](https://commons.wikimedia.org/wiki/File:HTTP_localhost.png), CC0, <https://commons.wikimedia.org/w/index.php?curid=60315543>



# Arbeiten mit TCP/IP: Client (**schematisch**)

```
import java.io.*;
import java.net.*;
class TCPCClient {
```

```
    public static void main(String argv[]) throws Exception
    {
```

```
        String sentence;
        String modifiedSentence;
```

```
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
```

Erstelle Client-  
Socket, baue  
Verbindung auf

```
        Socket clientSocket = new Socket("zielrechner", 6789);
```

Erstelle Ausgabe  
Datenstrom für  
den Socket

```
        DataOutputStream outToServer = new DataOutputStream(clientSocket.getOutputStream());
```

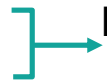
Angabe der Adresse des Servers in zwei Teilen: Adresse des Zielrechners (IP-Adresse/Name, der automatisch aufgeöst wird) und der auf diesem Rechner laufenden Anwendung, die die Daten bekommen soll (Port). Diese Adressinformationen werden mit einem sogenannten Socket verknüpft, der eine Variable zur Kommunikation per TCP/IP darstellt. Wir selber geben keinen Port an, der wird vom Betriebssystem gewählt





# Arbeiten mit TCP/IP: Client

Erstelle Lese  
Datenstrom  
aus dem  
Socket



```
BufferedReader inFromServer = new BufferedReader(new  
    InputStreamReader(clientSocket.getInputStream()));
```

```
sentence = inFromUser.readLine();
```

Sende an  
den Server



```
outToServer.writeBytes(sentence + '\n');
```

SEND

Empfange  
vom Server



```
modifiedSentence = inFromServer.readLine();
```

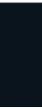
RECEIVE

```
System.out.println("FROM SERVER: " + modifiedSentence);
```

```
clientSocket.close();
```

```
}
```

```
}
```





# Arbeiten mit TCP/IP: Server (schematisch)

```
import java.io.*;  
import java.net.*;
```

```
class TCPServer {
```

```
    public static void main(String argv[]) throws Exception
```

```
    {  
        String clientSentence;  
        String capitalizedSentence;
```

Erstelle Socket

Warte auf  
eingehende  
Verbindungs-  
wünsche

Verknüpfe  
EingabeStream  
mit dem Socket

```
        ServerSocket welcomeSocket = new ServerSocket(6789);
```

```
        while(true) {
```

```
            Socket connectionSocket = welcomeSocket.accept();
```

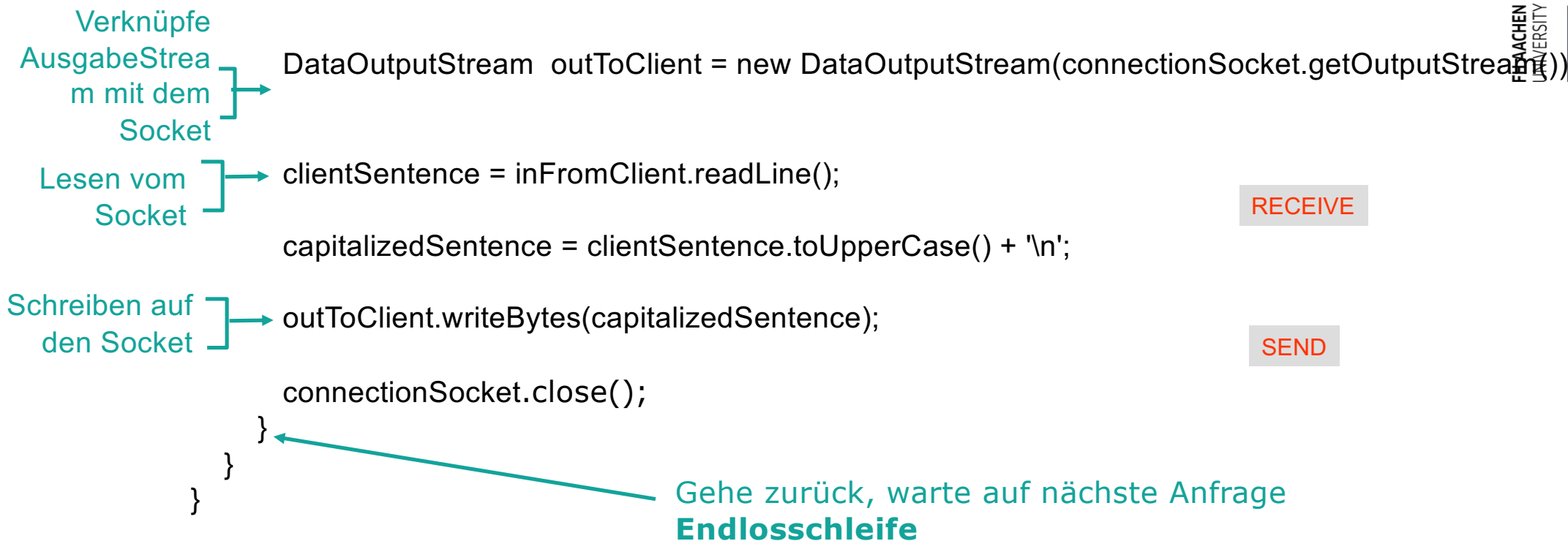
```
            BufferedReader inFromClient = new BufferedReader(new  
                InputStreamReader(connectionSocket.getInputStream()));
```

Wie beim Client: erstelle einen Socket als Variable zur Kommunikation. Aber: lege direkt fest, über welchen Port diese Anwendung Daten empfangen soll. Die IP-Adresse ist die IP-Adresse, des Rechners oder die festgelegte Standardadresse. Hier localhost!

Warte bis ein Client eine Verbindung über diesen Port aufbauen möchte



# Arbeiten mit TCP/IP: Server





## Socket und ServerSocket

---

- Um eine TCP-basierte Client-Server-Anwendung zu schreiben verwenden wir auf Server-Seite
  - Die **ServerSocket-Klasse**, mit der ein Server dem Betriebssystem anzeigen kann, unter welcher Port-Nummer Verbindungsanfragen eingehen können
  - Die **accept-Methode** der ServerSocket-Instanz wartet auf Verbindungsanfragen (blockiert) und liefert nach Aufbau der (logischen) Verbindung eine Instanz der Socket-Klasse zurück
  - Die **Socket-Klasse** bietet dann Methoden um einen Ein- und einen Ausgabe-Stream zu nutzen
- Auf Client-Seite arbeiten wir nur mit der Socket-Klasse
- Mit dem Schließen der Socket werden die assoziierten Streams ebenso geschlossen





---

Der eben implementierte Server hat ein Problem!

Welches?



### *Server-Seite*

- Der Anwendungsprozess, der Anfragen entgegennimmt (Server) muss zuerst laufen
- Der Server erstellt einen Socket, über den Verbindungsanfragen entgegengenommen werden (passive open, d.h. es wird ein Port bereitgestellt)
- **Um Anfragen mehrerer Clients entgegennehmen zu können, erstellt der Server bei einem Verbindungswunsch einen neuen Socket für den Client**

### *Client-Seite*

- Der Client erstellt einen Socket
- Der Client generiert eine Verbindungsanfrage
- Wird die Verbindung akzeptiert, so kann der Client mittels der erstellten Socket mit dem Server kommunizieren



Wenn ein Server mehrere Clients bedienen soll, so sind blockierende Leseoperation problematisch, denn ohne weitere Vorkehrung würde der Server ggf. auf eine Nachricht vom Client warten und bis diese eintrifft keine Verarbeitung anderer Nachrichten erlauben

TCP-Server werden somit entweder asynchron (Webengineering und Internet-Technologien) oder nebenläufig/parallel programmiert



# IT-Grundlagen Flashback: Prozesse und Threads

---

Ein Prozess ist die vom Betriebssystem genutzte Abstraktion eines in Ausführung befindlichen Programms

- Ein Prozess kann verschiedene **Zustände** haben
  - Rechnend
  - Wartend
  - Rechenbereit
- Jeder Prozess hat einen **eigenen Adressraum**
  - Keine gemeinsamen Variablen zwischen Prozessen
- Jeder Prozess arbeitet mit „eigenen“ Registern und hat insbesondere einen eigenen Programmzähler

Ein Thread ist ein *leichtgewichtiger* Prozess

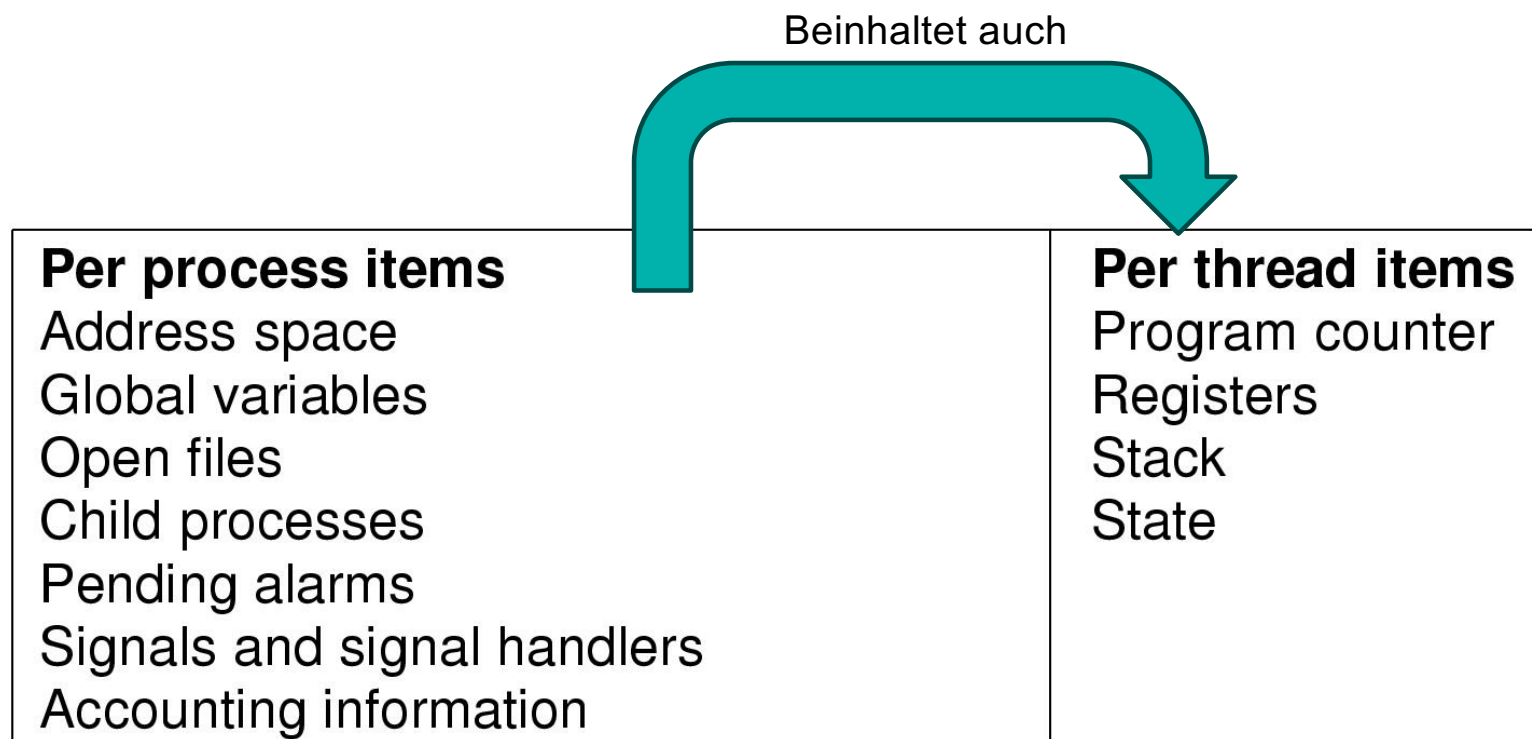
- Ein Thread gehört zu einem Prozess
- Alle Threads eines Prozesses teilen sich **denselben Adressraum**
  - Gemeinsame Variablen zwischen Threads möglich
- Ein Thread arbeitet mit „eigenen“ Registern und hat insbesondere einen eigenen Programmzähler







# Prozesse und Threads





## Entwicklung eigener verteilter Anwendungen

---

- Das Praktikum ist u.a. so konzipiert, dass Server-Anwendungen, die blockieren können über parallele/nebenläufige Abläufe verfügen müssen
- Um bei blockierenden Aufrufen trotzdem auch mit anderen Clients agieren zu können werden wir **Java-Threads** verwenden und zwar typischerweise immer dann, wenn wir eine Verbindung mit accept auf Server-Seite akzeptieren
- Threads waren bei einem „großen Programm“ schon ein Kernthema!
- Hinweis: Bei UDP blockiert man üblicherweise nicht, denn wir haben ja keine Idee einer „Verbindung“ sondern ein Briefkastenprinzip.
- Das Abarbeiten von Nachrichten eines beliebigen Absenders ist hier im Fokus. Faktisch kann man sich dies als eine Queue von Nachrichten vorstellen, die wir sukzessive bearbeiten

